Sovereign-V5-Anchor: System Report

Baseline ID: Sovereign-V5-Anchor **Sealed:** 2026-03-05 **Governance version:** v1 **Operator phase:** 5 **Sealed by:** Derek Angell

1. Executive overview

This document describes the current state of the SovereignNEXT system as of the sealing of its first authoritative baseline, Sovereign-V5-Anchor.

SovereignNEXT is a cognitive architecture that manages structured knowledge through claims, tensions between claims, and paradoxes that emerge from unresolved tensions. The system uses a local language model to generate new claims and detect tensions, then applies deterministic operators to enforce structural discipline on the results. A read-only observer inspects the system state after each pass and classifies what it finds.

On March 4, 2026, the system completed a canonical pipeline run: three passes of language-model-driven expansion followed by Phase 5 operator enforcement and sovereign observation. The run produced 650 claims, 1,505 tensions, and 84 paradoxes — all structurally bounded by the operator layer.

After the run completed, three governance contracts were created to formalize how the system reports on itself, how baselines are designated, and how governance itself evolves. The resulting final snapshot was then sealed as the first authoritative baseline under these contracts.

This document explains what existed before, what happened during the run, what was created, and why it matters. It is intended as a factual reference for technical review. It does not propose future work, make claims about capability, or prescribe next actions.

2. System state before this run

What existed

The SovereignNEXT system had been developed through a series of progressively more capable iterations:

- **V1** produced 76 claims and 36 tensions using a local language model. No paradoxes, no emoji vectors, no structural discipline beyond the LLM pipeline itself.
- **V2** introduced emoji vector mutation and paradox promotion. It produced 82 claims, 144 tensions, and 24 paradoxes. Polarity tensions emerged (87.5% of all tensions), confirming that the emoji vector bias was influencing tension detection. Hub concentration increased slightly.
- **V3** ran three Become passes starting from V2's final state. It produced 154 claims, 327 tensions, and 54 paradoxes. Hub diffusion was achieved — top-3 hub share dropped from 86.1% to 39.8%. Cross-family tension attachment was confirmed. All paradoxes remained in open status with no enforcement layer.

At this point, the system could expand and detect structure, but it had no mechanism to enforce boundaries on that expansion.

Phase 5 development

Phase 5 introduced three deterministic operators designed to constrain LLM-generated output:

- **Collapse** evaluates open tensions and either commits or defers them. If a tension involves a held paradox, it is held rather than committed.
- **Become** expands paradox emoji vectors mechanically and spawns derivative claims. It enforces an entropy ceiling (0.95) and a vector length limit (20).
- **Paradox-Hold** stabilizes paradox entropy within a target band (0.70–0.90), locks veto constraints, and records history events.

These operators were developed, unit-tested, and certified in isolation using controlled harnesses before being integrated with the LLM pipeline.

V5 experimental integration

An experimental integration run (`v5_integration_run.py`) combined the LLM pipeline with Phase 5 operators for the first time. That run started from the V3 final snapshot and completed three passes in approximately 107 minutes. It produced 644 claims, 477 tensions, and 84 paradoxes — all held, all vetoed, zero open tensions. This confirmed that Phase 5 operators could discipline fresh LLM output without operator failures.

What was missing

Despite the successful experimental run, several things did not yet exist:

- **No canonical pipeline.** The V5 integration runner was labeled "NON-CANONICAL / EXPERIMENTAL" and lived in the `experiments/` directory. There was no single authoritative execution path.
- **No governed observer semantics.** The sovereign observer detected anomalies but classified all oscillating paradoxes as warnings, even when the oscillation was a healthy expand-hold cycle. There was no distinction between regulated and pathological behavior.
- **No sealed baseline.** No snapshot had been explicitly designated as authoritative. There was no starting point for future runs that carried formal status.
- **No governance contracts.** Observer output had no vocabulary constraints, no severity contract, and no rules about what silence meant. There was no mechanism to prevent tools or interfaces from inventing meaning from observer output.
- **No lineage anchor.** The chain of V1 → V2 → V3 → V5 snapshots existed as files, but no formal lineage graph connected them.

3. The canonical V5 run

Promotion

Before running, the experimental integration script was promoted to canonical status. This involved:

- Copying `v5_integration_run.py` from `experiments/` to `SovereignNEXT/pipeline/run_sovereign_pipeline_v5.py`.
- Removing all "NON-CANONICAL / EXPERIMENTAL" labeling.
- Adding a configurable Phase 4/Phase 5 switch (defaulting to Phase 5).
- Making the starting snapshot, number of passes, and RNG seed configurable via command-line arguments.
- Redirecting all output artifacts to the `pipeline/` directory.
- Preserving identical execution logic — no behavioral changes.

The original experimental script was left untouched in `experiments/` for historical reference.

Execution

The canonical run was launched on March 4, 2026, at approximately 5:48 PM (UTC-5) and completed on March 5, 2026, at approximately 3:52 AM (UTC-5). Total duration: 604.4 minutes (approximately 10 hours).

Configuration:

- Starting snapshot: `v3_final_state_snapshot.json`
- Phase: 5
- Passes: 3
- Seed: 42

The extended duration compared to the experimental run (107 minutes) was due to a significantly larger tension field generated by the LLM. The tension detection step evaluates each new claim against all existing claims, and with more claims present, the number of pairwise comparisons grew substantially.

Per-pass results

Pass 1 — Broad expansion

Metric	Before	After	Delta
Claims	154	178	+24
Tensions	327	388	+61

Metric	Before	After	Delta
Paradoxes	54	64	+10

Phase 5: 64 paradoxes held, 64 vetoed. 388 tensions held by Collapse (0 committed). Become expanded all 64 eligible paradoxes, spawning 128 derivative claims. Hold stabilized all 64.

LLM duration: 2,737s. Phase 5 duration: 0.14s.

Pass 2 — Targeted expansion

Metric	Before	After	Delta
Claims	306	330	+24
Tensions	388	696	+308
Paradoxes	64	74	+10

Phase 5: 74 held, 74 vetoed. Become expanded 64, stabilized 10 (newly promoted paradoxes with vectors at entropy ceiling). Hold nudged 10 paradoxes to bring entropy into band.

LLM duration: 10,957s. Phase 5 duration: 0.07s.

Pass 3 — Adaptive expansion

Metric	Before	After	Delta
Claims	458	482	+24
Tensions	696	1,505	+809
Paradoxes	74	84	+10

Phase 5: 84 held, 84 vetoed. Become expanded all 84. Hold stabilized all 84. Sovereign observer detected 68 anomalies.

LLM duration: 22,570s. Phase 5 duration: 0.14s.

Final state

Metric	V3 Baseline	Canonical Final	Delta
Claims	154	650	+496
Tensions	327	1,505	+1,178
Paradoxes	54	84	+30
Emoji vectors	54	84	+30
Open tensions	327	0	-327
Held paradoxes	0	84	+84
Vetoed paradoxes	0	84	+84

Tension type breakdown: 1,487 polarity (98.8%), 11 contradiction (0.7%), 7 tradeoff (0.5%).

Hub concentration: top-3 claims hold 25.3% of tension references, down from 39.8% in V3 and 86.1% in V2.

Content hash (input):
`48e2859d1e3e16a0ac23b132a9564f7b1ab4ef2a8b497b414accf95826607b79`
Content hash (final):
`f9a12fa44008c6998943066d332811971c1223f4261d4209810ee3eb61040bea`

4. Observer governance and meaning control

The problem

The sovereign observer inspects system state and emits classifications. Without governance, those classifications can be misinterpreted:

- A "warning" could be read as requiring action when it is simply descriptive.
- Silence could be read as either health or absence of observation.
- An interface or tool could auto-react to observer output, inventing urgency that does not exist.

- Observer language could drift over time, changing meaning without changing code.

These are not hypothetical risks. They are the default behavior of any reporting system that lacks explicit semantic constraints.

What was done

An **Observer Governance Contract (v1)** was created to constrain observer behavior at the semantic level. This contract specifies:

Vocabulary limits. Observer messages must not include words like "should," "recommend," "consider," "next," "optimize," or "trigger." Observer messages must not instruct actions, propose plans, assign blame, imply urgency, or imply authority over execution. Observer messages may describe what was detected, cite evidence, state classification and severity, and state uncertainty when evidence is incomplete.

Severity semantics. Two severity levels exist: `info` and `warning`. The mapping is fixed:

- `regulated` → info
- `stuck`, `saturated`, `oscillating`, `drifting` → warning

No other severities exist in v1.

Silence interpretation. Silence (zero warnings) is a positive health signal only when the observer ran successfully and artifacts were produced. Silence is not interpreted as health when the observer did not run or the report is missing. Any interface must distinguish between healthy silence and missing observation.

Health summary rules. Every run produces a single health summary object with fixed fields: anomalies total, warnings total, regulated total, warnings by type, and a health statement. The health statement is binary: either "healthy: no warnings" or "warnings present: review anomalies." No other health statements exist in v1.

Non-authority. The observer has zero execution authority. It cannot mutate state, trigger operators, decide next actions, or block runs. It has descriptive authority only.

Why this matters

Observer governance means that observer output can now be surfaced by any tool or interface without risk of semantic drift. The meaning of every classification, every severity level, and every silence condition is defined once and constrains all downstream interpretation.

5. Health summary and warnings

The canonical run's health summary


```
{
  "anomalies_total": 68,
  "regulated_total": 60,
  "warnings_total": 8,
  "warnings_by_type": {
    "oscillating": 4,
    "drifting": 4
  },
  "health_statement": "warnings present: review anomalies"
}
```


What the 60 regulated anomalies are

Sixty paradoxes exhibit an alternating expand-hold cycle — Become expands the emoji vector, then Paradox-Hold stabilizes entropy back into the target band. This alternation is detected by the observer and classified as `regulated` because all six criteria for healthy oscillation are met:

1. Entropy remains within the configured band (0.70–0.90).
2. Entropy drift between consecutive hold events is within threshold (|delta| ≤ 0.05).
3. Collapse veto is intact.
4. Paradox status is `paradox_held` .
5. Emoji vector is not saturated.
6. No monotonic entropy rise across three or more hold events.

These are informational signals. They confirm that the expand-hold cycle is functioning as designed.

What the 8 warnings are

The 8 warnings are concentrated on four paradoxes: `paradox_0001` through `paradox_0004`. These are the oldest paradoxes in the system, originating from V2. Each receives two flags:

- **oscillating (4):** Alternating pattern detected with a pathological signal — entropy has dropped below the band floor (0.70).
- **drifting (4):** Monotonic entropy decline across the last three hold events: 0.7264 → 0.7061 → 0.6933.

These paradoxes have accumulated the most history (8 events each) and the longest emoji vectors (length 37). Their entropy has drifted below the stabilization band floor through repeated mutation cycles.

Why warnings do not block authority

The baseline sealing governance contract explicitly states that eligibility for sealing does not depend on observer severity counts, absence of warnings, entropy values, or novelty metrics. Sealing is a human act based on deliberate judgment, not a metric threshold.

The warnings are preserved because they are real. The four oldest paradoxes are exhibiting measurable entropy compression. That is a structural observation worth recording, not a defect to suppress.

For comparison: paradoxes `0005` through `0008`, which have the same history depth but slightly shorter emoji vectors (length 34), show entropy of 0.7227 — inside the band, classified as regulated.

6. Governance contracts created

Three governance contracts were created in `SovereignNEXT/governance/` after the canonical run completed. These contracts are normative documents, not documentation. They define constraints that other components must obey.

Observer Governance Contract v1

Governs: What the observer is allowed to say and how its output is interpreted.

Defines: anomaly types and severity mapping, evidence schema, vocabulary constraints, forbidden and allowed speech acts, silence semantics, health summary schema, and OpenClaw integration rules.

Key principle: The observer has zero execution authority. It is a reporter, not a controller.

Baseline Sealing and Lineage Governance Contract v1

Governs: When and how a pipeline artifact becomes authoritative.

Defines: what a baseline is (a deliberate designation, not a metric outcome), eligibility rules, required sealing metadata, lineage preservation rules, immutability constraints, revocation rules, and interface constraints.

Key principle: Only a human may designate or revoke a baseline. No tool may auto-seal, recommend sealing, or imply readiness for sealing.

Governance Proposal and Ratification Contract v1

Governs: How governance itself evolves.

Defines: governance states (proposed vs. ratified), proposal requirements, ratification process (requires document + canonical run + baseline seal + metadata record), authority boundaries, and backward compatibility rules.

Key principle: Governance may evolve, but never implicitly. No meaning changes unless it is written, versioned, exercised in a canonical run, and sealed by a human.

How these contracts interact

The three contracts form a closed governance loop:

- The **observer contract** constrains meaning.
- The **sealing contract** constrains authority.
- The **ratification contract** constrains change.

Any modification to any of these layers requires a new version document, a canonical run referencing it, and a baseline sealed under it. This prevents silent semantic drift.

7. Baseline sealing and naming

What sealing means

Sealing designates a specific snapshot as the authoritative starting state for future canonical runs. It is not an assertion of quality, health, or completeness. It is a deliberate act that says: "This is the system we are choosing to stand on."

The sealing act

The snapshot `v5_final_state_snapshot.json` was sealed as `Sovereign-V5-Anchor` with the following metadata, stored in `Sovereign-V5-Anchor.seal.json` adjacent to the snapshot:

Field	Value
baseline_id	Sovereign-V5-Anchor
snapshot_hash	`f9a12fa44008c699...`
origin_run_id	Canonical V5 Pipeline Run
origin_run_timestamp	2026-03-04T22:47:55Z
pipeline_version	run_sovereign_pipeline_v5.py
governance_version	v1
operator_phase	5
passes	3
seed	42
sealed_at	2026-03-05T09:41:00Z
sealed_by	Derek

Why "Sovereign-V5-Anchor"

The name communicates three things:

- **Sovereign** — this is the first snapshot sealed under formal governance.
- **V5** — it was produced by the Phase 5 operator architecture.
- **Anchor** — it serves as the fixed reference point for future runs, not as a claim of finality or perfection.

The name remains correct regardless of what future baselines contain.

Immutability

Per the sealing contract, this baseline is immutable. The metadata cannot be edited in place. If a correction is needed, it requires a new baseline designation. The snapshot itself is never modified after sealing.

Lineage

The sealed baseline records its origin: `v3_final_state_snapshot.json` with hash `48e2859d1e3e16a0...`. This creates the first link in a formal lineage graph:

```
V1 → V2 → V3 → Sovereign-V5-Anchor
```

Future canonical runs that start from this baseline will record its hash, extending the chain.

8. How this differs from earlier versions

No auto-optimization

The system does not optimize for any metric. Phase 5 operators enforce structural constraints (entropy bounds, veto locks, vector length limits), but they do not pursue improvement. The observer reports what it finds; it does not steer toward better outcomes.

Earlier versions had no enforcement layer at all. The LLM pipeline expanded freely, and structural properties like hub concentration were observed after the fact but not governed during execution.

No metric chasing

The 8 warnings in this run were preserved, not suppressed. The baseline was sealed with warnings present. Under the governance contract, health does not determine sealability. This is a deliberate architectural choice: the system records truth about itself rather than optimizing for the appearance of health.

No implicit authority

In earlier versions, there was no distinction between an experimental run and an authoritative one. Snapshots accumulated without formal status. The observer existed but had no constraints on how its output could be interpreted.

Now, authority is explicit at every layer:

- The pipeline is labeled canonical.
- The observer's language is constrained by contract.
- Baselines require human designation.
- Governance requires human ratification.

No retroactive reinterpretation

Governance contracts are immutable once published. Older versions remain valid for historical runs. No governance version may retroactively reinterpret past artifacts. This means that the meaning of Sovereign-V5-Anchor is fixed at the time of sealing, regardless of how the system evolves afterward.

No silent drift

Changes to observer vocabulary, severity semantics, silence interpretation, health summary schema, baseline eligibility rules, or authority boundaries all require a new governance version document. The ratification contract ensures that governance evolves only through lived execution (canonical run + baseline seal), not through theory or proposal alone.

9. What this enables next

The following capabilities are now available. None are committed or scheduled. They are listed as structural possibilities, not plans.

Future canonical runs can reference a sealed baseline. Any run starting from Sovereign-V5-Anchor inherits a known, stable, hash-verified starting state with complete lineage.

Governance can evolve deliberately. If observer semantics, severity mappings, or eligibility rules need to change, the ratification contract defines exactly how: propose a new version, run canonically under it, seal a baseline, record the governance version in the metadata.

Longitudinal behavior can be observed. The four drifting paradoxes (0001–0004) now have a recorded trajectory. Future runs will show whether their entropy continues to compress, stabilizes, or reverses. This signal is preserved, not erased.

The system can rest without decaying. The sealed baseline, governance contracts, and canonical pipeline are all static artifacts. They do not require maintenance, monitoring, or periodic execution to remain valid.

Interfaces can surface governed output. Any presentation layer (CLI, web UI, conversational interface) can now read observer output, display health summaries, list anomalies, and show lineage — all within the constraints defined by the governance contracts, without inventing meaning.

Appendix: Artifact inventory

SovereignNEXT/pipeline/

File	Description
`run_sovereign_pipeline_v5.py`	Canonical pipeline entrypoint
`Sovereign-V5-Anchor.seal.json`	Baseline sealing metadata

File	Description
`v5_final_state_snapshot.json`	Sealed baseline snapshot
`v5_pass1_state_snapshot.json`	Pass 1 intermediate snapshot
`v5_pass2_state_snapshot.json`	Pass 2 intermediate snapshot
`v5_pass3_state_snapshot.json`	Pass 3 intermediate snapshot
`v5_canonical_report.json`	Run report with health summary

SovereignNEXT/governance/

File	Description
`observer_governance_contract_v1.md`	Observer output and meaning constraints
`baseline_sealing_governance_contract_v1.md`	Baseline designation and lineage rules
`governance_ratification_contract_v1.md`	Governance evolution mechanics

SovereignNEXT/experiments/ (historical, unchanged)

File	Description
`v5_integration_run.py`	Original experimental integration runner
`v5_anomaly_forensics.py`	Anomaly analysis script
`sovereign_oscillation_classification.md`	Observer refinement design spec

This document was generated from canonical artifacts on 2026-03-05. All numbers are derived from actual pipeline output. No values are projected, estimated, or aspirational.